

Generative Modeling Case Study by Sandeep Kumar 23095447

[Github](#)

Introduction

In this assignment, the objective is to build a generative adversarial network that is split into 2 parts where part 1 is an extension of the tutorial that explains the general concept of working of a GAN followed by part 2. Part 2 is split into 3 parts where we use these GAN to address real world problems, that is Medicine, Cybersecurity and creative arts.

Methodology

We use a simple generative adversarial network for this case study as we know a GAN works on 2 neural network, a generator that generates the data from random noise and a discriminator that tries to prove whether the data is fake or not. They work on sum 0 formula that makes it quite easy to configure and generate good fake results. (Tensorflow, 2024) In some areas we are also using DCGAN which is specifically designed to generate images, it's a type of vanilla GAN FC layer with CNN layer, making it better in image generating. In this project most of our objectives requires us to generate images to using a DCGAN is a better option.

Part 1: Generating a 2D spiral using GAN

We start by creating a simple 2-D spiral with 10 rotations and 4096 data points for consistency. Unlike the tutorial where we had to generate a simple linear sine wave, generating a spiral is significantly tough as it's a multi-dimensional, non-linear structure. A 2-d spiral requires a model to understand angle and distance because it expands and rotates at the same time.

After implementing GAN with default parameters given in tutorial, the results are given below (figure 1). The model failed to capture even the basic spiral pattern and resulted in a mode collapse.

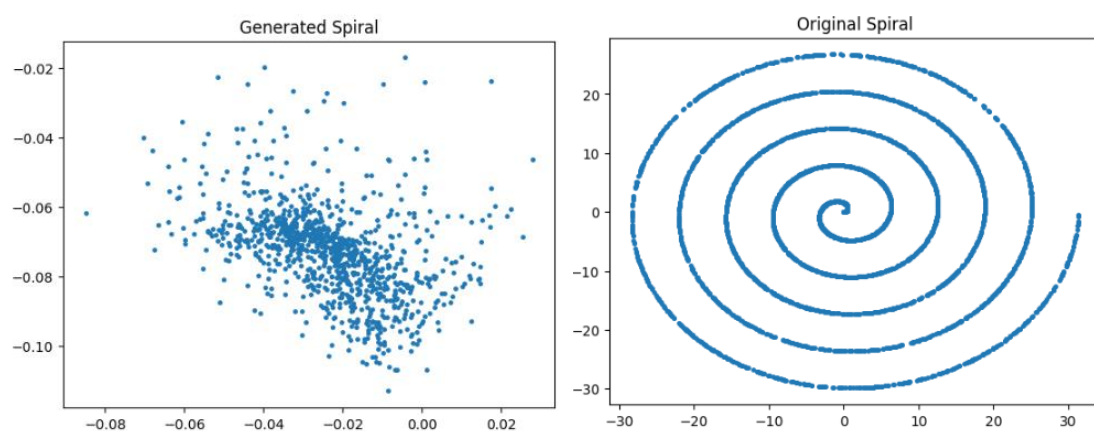


Figure 1: 2-D spiral Original vs generated samples

To improve this, several trial and error steps were taken, that included reducing the number of spirals and data points, normalizing the data, changing the loss function from BCE to BCEwithlogitsloss and switching activation function from ReLU to LeakyReLU, that made significant difference to the entire training process. Other smaller hyperparameter adjustments were also made to ensure a good result, the output us given below.

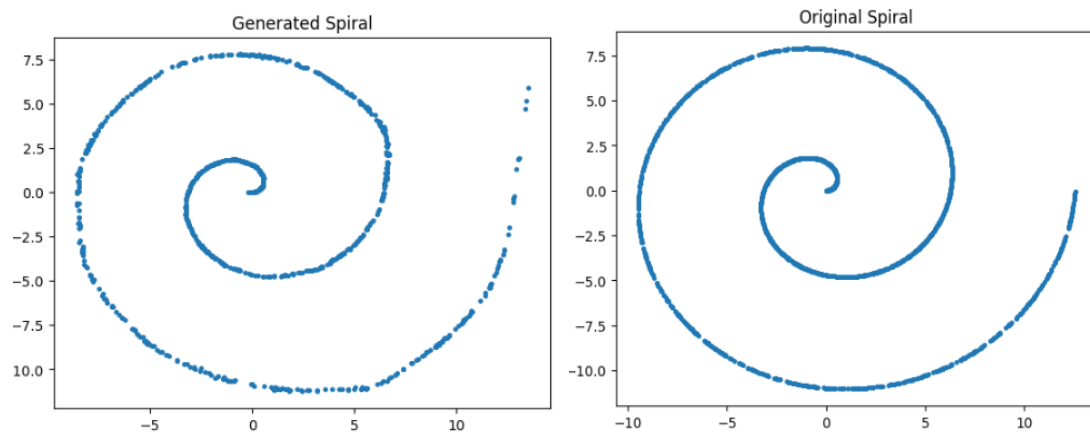
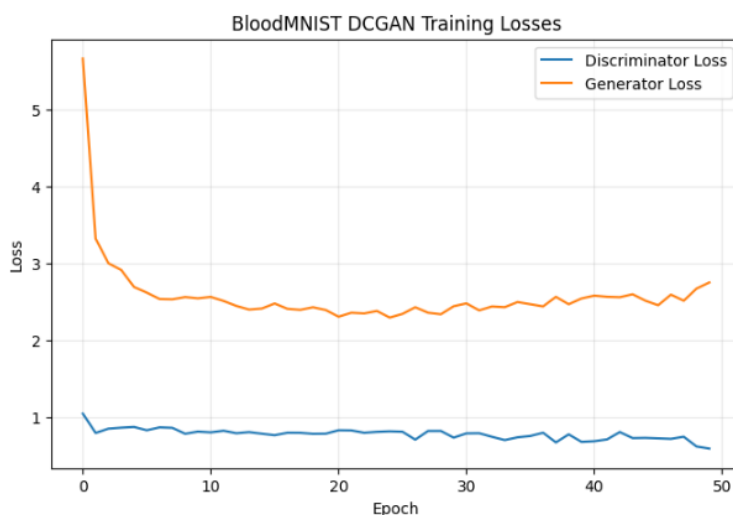


Figure 2: 2-D Spiral Original vs generate after tuning

Part 2.1: Blood cells sample image generation

For our next task, we will attempt to generate blood cells samples using the dataset bloodMNIST, which comprises labelled microscopic image of blood cells categorized in 8 different classes. We will use DCGAN (Deep Convolutional Generative Adversarial Network) to train this dataset. DCGAN is known to effective against leearning spacial features like texture pattern and colour distribution mostly present in cellular images. After training the data we can summaries the training performance with a graph given below. (Figure 3: Training sequence)



As we can see from the the discriminator is approximately at 0.7 units and stays stable throughout training, the generator starts high and then stabilizes around 2.3 - 2.7 units. We can deduct a few points from this, firstly the discriminator is a bit strong and hard to fool, the generator drops, flattens and slightly

improves at the end which is normal GAN behaviour, not very strong though, however there are no anomalies like mode collapse, making the training stable.

After training a comparison of original data and final data is given below.

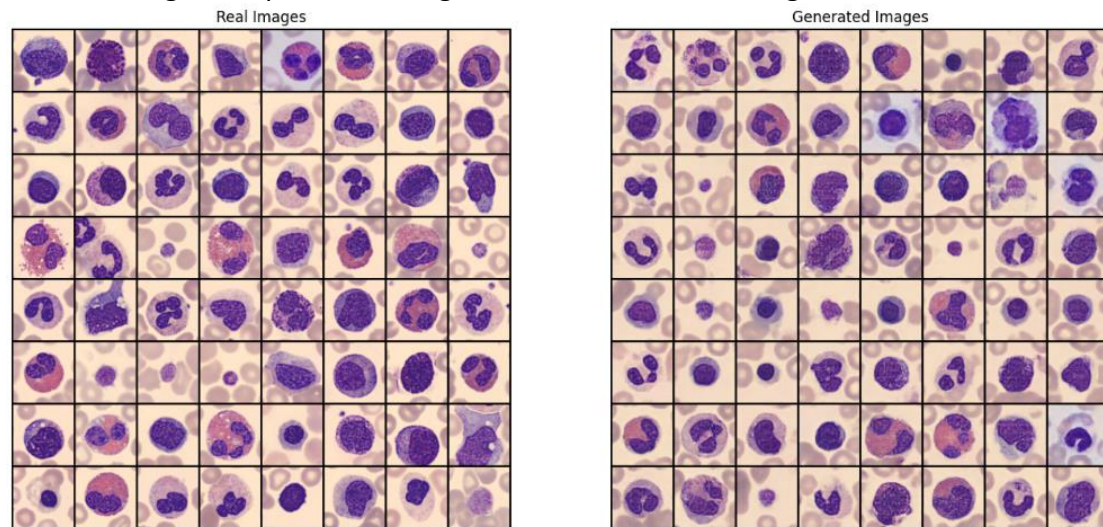


Figure 4: Real vs Original cells data

The generated image has good circular cells structure with purple staining patterns and nucleus like region. However the details are sharply lacking and some images are a bit blurry, which make sense as as learning curve was slight off. There also has been a mild mode collapse in here as some of the samples have repeated looking structure.

Possible solution for improvement would be to make discriminator a bit weak and use a better GAN variant. The FID score of 54.52 suggest that the performance was acceptable at best and can be significantly improved with the help of stronger computing power device that might help in trial and error scenarios.

Part 2.2: Synthetic Traffic with CICIDS

We are using a standard GAN for this task to generate cybersecurity synthetic rerecords CICIDS 2017 dataset. The dataset is a network intrusion detection dataset, that simulates real world network traffic. Each file corresponds to different type of traffic where Monday is mostly benign, Wednesday files include attacks like DoS and heartbleed. Thursday files focuses on web sites type attacks that are generally SQL injection, brute force login and XXS attacks and Firday files has DDoS attack which is a modified version on DoS attack.

The most important columns we can find here are benign, DDoS, hulk/golden eye, SQL injection and brute force. After labeling and scaling the data we can use it in a GAN model to generate synthetic traffic. The data is numerical so instead of a DCGAN a standard GAN is more suitable. The generated data is in vector form that can be compared with real data using PCA. The training sequence and comparison is given below.

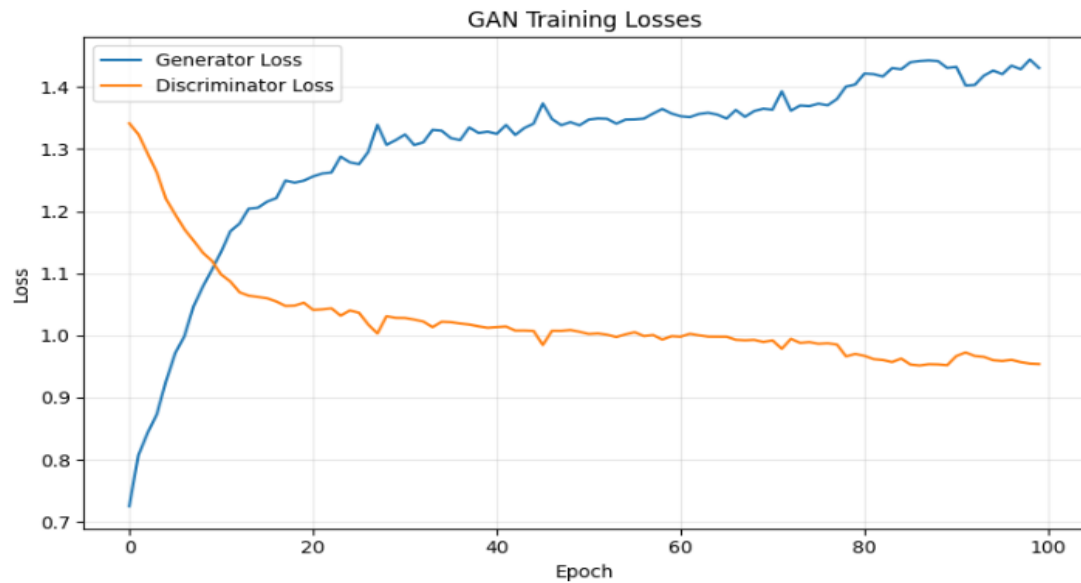


Figure 5: GAN loss throughout epochs

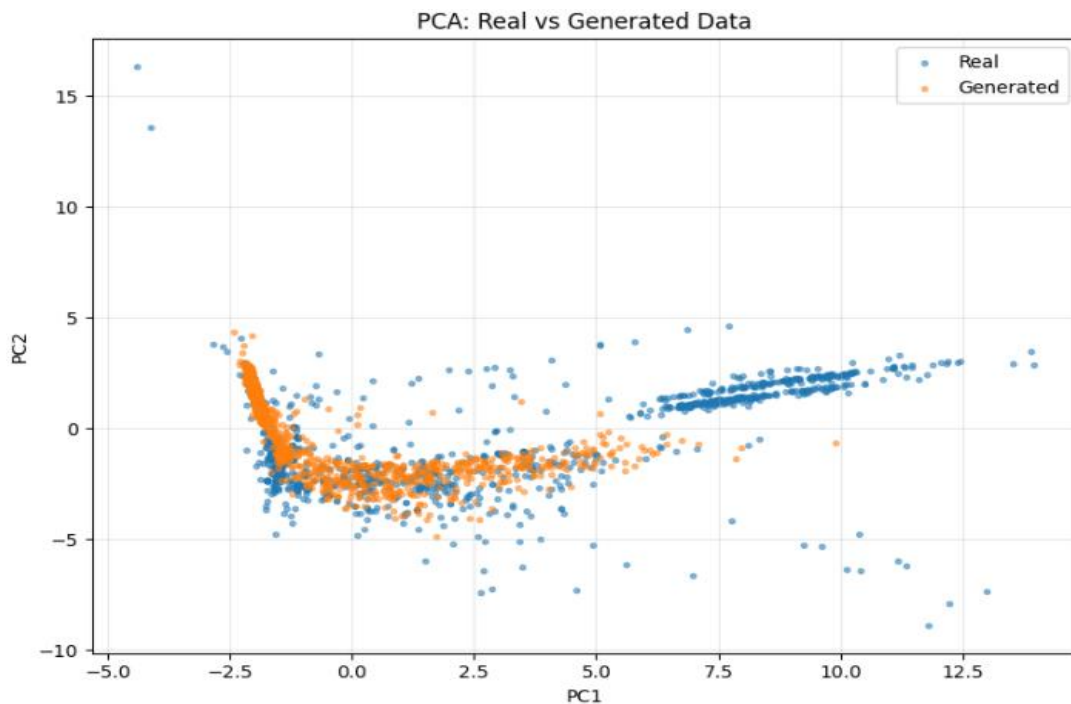
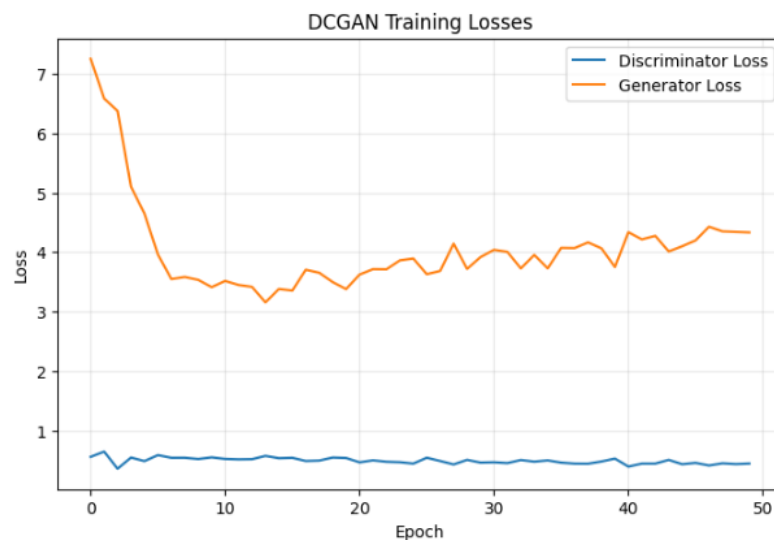


Figure 6: Original vs generated data cybersec data

From the result we can say there is an averagely moderate overlap between the original and generated data. The GAN is struggling to capture the complete data distribution however it is capturing partial structure properly. A lot of improvement can be made by either switching to a better GAN model, improving generator capacity or balancing the generator training by training generator twice and discriminator one in every epochs. Overall the performance is just average.

Part 2.3: Pizza Sketch generation

Finally, using the quick-draw pizza sketches to train a DCGAN model in the same manner as blood samples, but this time data is a bit different. Earlier we have clear cell structures that were photographic coloured images unlike hand-drawn images like we have now. These images are slightly easier to train and level of details are low here.(Figure 7)



From this final task its clear that discriminator is bit dominant when kept under a generator with mostly default settings. But in this scenario the discriminator seems to be mitigating over time which is positive sigh. On the other hand the generator which drops

immediately and then slowly increases with the number of epochs suggest that the training was stable with no mode collapse.

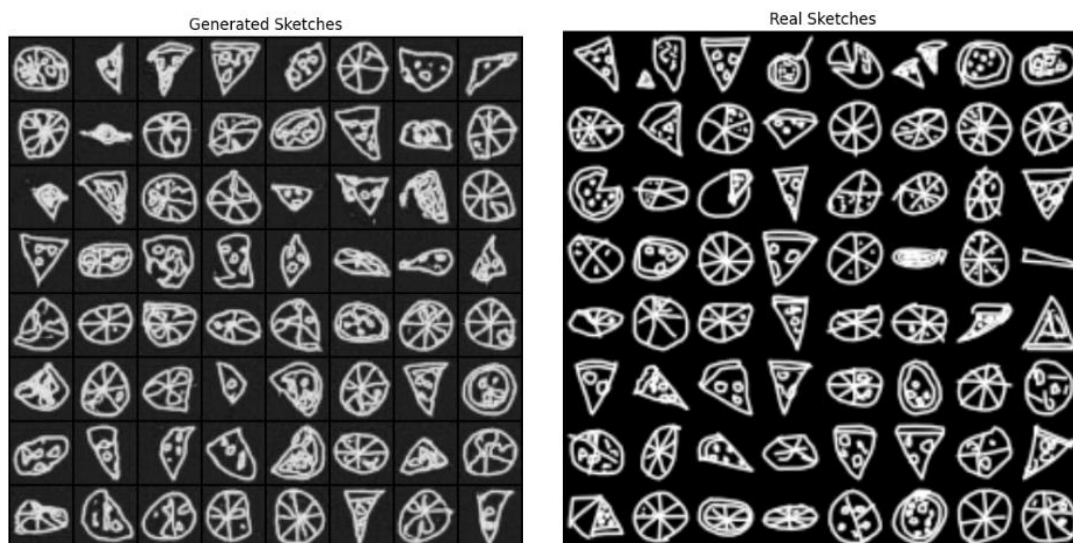


Figure 8: Real vs generated pizza data

We can see the generator clearly learned the circular shapes, slice like structure and toppings and many outputs look like pizza. There is a slight blurry-ness in the generated sample and there are a few sample with distorted geometry. Learning strstructure is not perfect but overall model performance is not bad.

Conclusion and improvements

Across the 3 tasks the results were both good and bad when applied on different type of data. In the standard GAN (cybersecurity) the training and result shows a clear imbalance as discriminator outperformed generator in many instances which resulted in average quality of synthetic data. This could mean the the PCA visualization failed to capture full distribution of data and collapsed instead.

Unlike the standard GAN, DCGAN trained on blood and pizza data demonstrated better performance, with minimum model collapse and smoother training. Although the loss curve still indicates a dominant discriminator, the generator model still managed to capture key details like circular shape, slice pattern, cells structure. The output mostly suffered from blurriness, limited diversity and minor distortions suggesting that model did not learn properly and there was a mild mode collapse.

As for improvements, a standard GAN is proven to be a bit weak, exploring better versions of GAN might improve the performance, a balanced generator and discriminator will also help here. Improving model architecture and better preprocessing can lead to a good model.

References

1) GAN and DCGAN

[Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D.,](#)

[Ozair, S., Courville, A., & Bengio, Y. \(2014\).](#)

[Generative Adversarial Nets. NeurIPS.](#)

2) [PyTorch. \(n.d.\). DCGAN Tutorial.](#)

3) [Hall, J. E. \(2020\). Guyton and Hall Textbook of Medical Physiology \(14th ed.\)](#)

4) [Mescher, A. L. \(2021\). Junqueira's Basic Histology: Text and Atlas \(16th ed.\)](#)